

Universidad de San Carlos de Guatemala  
Facultad de Ingeniería  
Ingeniería en Ciencias y Sistemas



# AutoInfra

## Lenguaje interpretado para automatización y simulación de infraestructura

### Proyecto 2 - Organización de Lenguajes y Compiladores 1

Ponderación: 55 pts

Fecha de entrega: 26 de octubre 2026

## Índice

|          |                                                  |
|----------|--------------------------------------------------|
| 1. ....  | Objetivos                                        |
| 2. ....  | Descripción                                      |
| 3. ....  | Introducción                                     |
| 4. ....  | Estructura del lenguaje                          |
| 5. ....  | Sistema de tipos                                 |
| 6. ....  | Variables, expresiones y operadores              |
| 7. ....  | Estructuras de control                           |
| 8. ....  | Funciones y ámbitos                              |
| 9. ....  | Recursos de infraestructura                      |
| 10. .... | Acciones nativas de AutoInfra                    |
| 11. .... | Tareas y automatización                          |
| 12. .... | Dependencias y reglas de ejecución               |
| 13. .... | Patrón Interpreter y AST                         |
| 14. .... | Análisis léxico, sintáctico y semántico          |
| 15. .... | Punto de entrada                                 |
| 16. .... | Entorno de trabajo                               |
| 17. .... | Reportes                                         |
| 18. .... | Requerimientos mínimos                           |
| 19. .... | Ponderación                                      |
| 20. .... | Entregables                                      |
| 21. .... | Restricciones                                    |
| 22. .... | Casos de prueba sugeridos                        |
| 23. .... | Fecha de entrega                                 |
| 24. .... | Arquitectura cliente-servidor e interfaz gráfica |
| 25. .... | Herramientas permitidas para el análisis         |

|          |                                             |
|----------|---------------------------------------------|
| 26. .... | Operaciones permitidas                      |
| 27. .... | Propiedades disponibles por<br>recurso      |
| 28. .... | Catálogo de expresiones con<br>ejemplos     |
| 29. .... | Funciones nativas: ejemplos<br>obligatorios |
| 30. .... | Ejemplo integral de AutoInfra               |
| 31. .... | Tecnologías definitivas                     |

## 1. Objetivos

### 1.1 Objetivo general

Aplicar los conocimientos de análisis léxico, sintáctico y semántico, construcción de árboles de sintaxis abstracta y ejecución mediante el patrón de diseño Interpreter, desarrollando en JavaScript un lenguaje propio capaz de describir, automatizar y simular infraestructura tecnológica.

El sistema deberá analizar archivos con extensión .infra, construir un AST, validar tipos y referencias, y ejecutar instrucciones que modifiquen un entorno de infraestructura completamente simulado.

### 1.2 Objetivos específicos

- Construir el analizador léxico y sintáctico en JavaScript, permitiendo el uso de ANTLR u otras librerías/generadores de analizadores.
- Construir un analizador sintáctico mediante descenso recursivo u otra técnica manual aprobada.
- Diseñar una gramática no ambigua con precedencia y asociatividad definidas.
- Construir un AST y aplicar el patrón Interpreter para ejecutar sus nodos.
- Implementar tabla de símbolos con ámbitos globales, locales y de función.
- Implementar tipos de datos, variables, arreglos, operadores aritméticos, relacionales y lógicos.
- Implementar estructuras if/else, while y for, además de break, continue y return.
- Implementar funciones con parámetros, valores de retorno y recursividad.
- Modelar servidores, servicios y bases de datos como recursos del lenguaje.
- Simular operaciones de despliegue, instalación, escalamiento y conexión entre recursos.
- Detectar y reportar errores léxicos, sintácticos y semánticos.
- Generar reportes del AST, tabla de símbolos, errores y estado final de infraestructura.

## 2. Descripción

El estudiante deberá desarrollar AutoInfra, un intérprete de un lenguaje de dominio específico (DSL) orientado a automatización de infraestructura. El lenguaje tomará inspiración conceptual de herramientas de Infrastructure as Code, pero no deberá conectarse a proveedores cloud, contenedores, sistemas operativos ni servidores reales.

La ejecución será una simulación determinista: las instrucciones modificarán estructuras internas en memoria y producirán una bitácora de cambios. Esto permite concentrar la evaluación en compiladores, estructuras de datos, semántica y patrón Interpreter.

## 3. Introducción

### 3.1 ¿Qué problema resuelve AutoInfra?

AutoInfra permitirá expresar infraestructura y procedimientos de automatización en un lenguaje compacto. Un archivo podrá declarar recursos, definir funciones y tareas, evaluar condiciones y ejecutar ciclos para administrar múltiples recursos.

```
server backend {  
  cpu = 4;  
  memory = 8;
```

```
    disk = 100;
    os = "ubuntu";
}

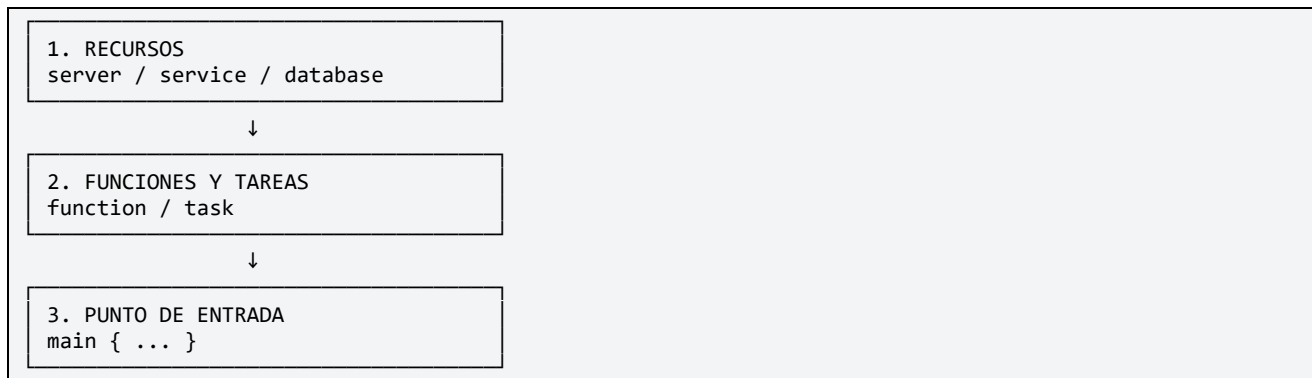
service api {
    port = 8080;
    replicas = 2;
}

task production {
    install(backend, "docker");
    start(backend);
    deploy(backend, api);
    scale(api, 3);
}

main {
    run production;
}
```

## 4. Estructura del lenguaje

Un archivo AutoInfra estará compuesto por declaraciones globales y un único punto de entrada main. Las declaraciones podrán aparecer antes de main en cualquier orden permitido por la gramática.



### 4.1 Comentarios

Los comentarios deberán ser ignorados por el analizador léxico.

| Tipo       | Inicio | Fin            | Ejemplo               |
|------------|--------|----------------|-----------------------|
| Una línea  | //     | salto de línea | // servidor principal |
| Multilínea | /*     | */             | /* configuración */   |

Un comentario multilínea sin cierre deberá producir un error léxico indicando línea y columna.

### 4.2 Identificadores

Los identificadores comenzarán con una letra o guion bajo y podrán continuar con letras, dígitos o guion bajo. Son sensibles a mayúsculas y minúsculas.

```

backend
_backend2
apiProduccion

// Identificadores diferentes:
Backend
backend

```

### 4.3 Palabras reservadas

| Categoría     | Palabras                                            |
|---------------|-----------------------------------------------------|
| Tipos         | int, float, string, bool, server, service, database |
| Control       | if, else, while, for, break, continue, return       |
| Declaraciones | function, task, main                                |
| Ejecución     | run                                                 |
| Literales     | true, false                                         |

## 5. Sistema de tipos

AutoInfra tendrá tipado estricto. No se permitirá asignar un valor incompatible sin una conversión explícitamente definida por el proyecto.

| Tipo     | Descripción                | Ejemplo           |
|----------|----------------------------|-------------------|
| int      | Número entero              | 10                |
| float    | Número decimal             | 2.5               |
| string   | Cadena de caracteres       | "production"      |
| bool     | Valor lógico               | true              |
| server   | Referencia a servidor      | backend           |
| service  | Referencia a servicio      | api               |
| database | Referencia a base de datos | postgres          |
| T[]      | Arreglo homogéneo          | string[] packages |

### 5.1 Valores por defecto

| Tipo        | Valor por defecto   |
|-------------|---------------------|
| int         | 0                   |
| float       | 0.0                 |
| string      | ""                  |
| bool        | false               |
| referencias | null lógico interno |

### 5.2 Reglas de compatibilidad

- int puede participar en operaciones con int.
- float puede participar con float y con int; el resultado numérico será float cuando exista al menos un float.
- string admite concatenación mediante +.
- bool se utiliza en condiciones y operadores lógicos.
- Los recursos solo pueden compararse por igualdad o desigualdad de referencia.
- Los arreglos deben contener elementos de un mismo tipo.
- Una condición de if, while o for debe producir bool.

## 6. Variables, expresiones y operadores

```
int replicas = 3;
float cpuLimit = 2.5;
string environment = "production";
bool active = true;

int total = replicas * 2 + 1;
bool ready = total >= 5 && active;
string node = "worker-" + replicas;
```

### 6.1 Operadores aritméticos

| Operador | Operación            |
|----------|----------------------|
| +        | suma / concatenación |
| -        | resta                |
| *        | multiplicación       |
| /        | división             |
| %        | módulo               |

### 6.2 Operadores relacionales y lógicos

| Grupo        | Operadores           |
|--------------|----------------------|
| Relacionales | ==, !=, >, <, >=, <= |
| Lógicos      | &&,   , !            |

### 6.3 Precedencia

1. Paréntesis, llamadas, acceso a propiedades e índices.
2. Negación lógica ! y negación aritmética -.
3. \*, /, %.
4. + y -.
5. <, <=, >, >=.
6. ==, !=.
7. &&.
8. ||.

Los operadores && y || deberán implementar evaluación de corto circuito.

## 7. Estructuras de control

### 7.1 Condicional if/else

```
if (backend.memory >= 8 && backend.status == "running") {
    print("Servidor disponible");
} else {
    print("Servidor no disponible");
}
```

El bloque else será opcional. Se permitirán if anidados y cadenas else if si la gramática del estudiante las descompone como un else que contiene otro if.

### 7.2 Ciclo while

```
int attempts = 0;

while (attempts < 3) {
    restart(backend);
    attempts = attempts + 1;
}
```

### 7.3 Ciclo for

```
for (int i = 0; i < 3; i = i + 1) {
    print("Configurando nodo " + i);
}
```

### 7.4 break y continue

```
for (int i = 0; i < length(nodes); i = i + 1) {
    if (nodes[i].status == "failed") {
        continue;
    }

    start(nodes[i]);

    if (i == 10) {
        break;
    }
}
```

break y continue utilizados fuera de un ciclo deberán registrarse como errores semánticos.

### 7.5 Límite de seguridad

Para evitar bloqueos por ciclos infinitos durante la calificación, el intérprete deberá permitir configurar un máximo de 100,000 instrucciones ejecutadas. Al superar el límite, la ejecución se detendrá con un error de ejecución controlado.

## 8. Funciones y ámbitos

Las funciones permiten encapsular lógica reusable. Cada llamada deberá crear un nuevo entorno local enlazado al entorno donde la función fue declarada.

```
function canDeploy(server s, int minMemory) bool {
    if (s.memory >= minMemory && s.status == "running") {
        return true;
    }
    return false;
}
```

### 8.1 Parámetros y retorno

- Los parámetros tendrán nombre y tipo.
- El número y tipo de argumentos deberá coincidir con la firma.
- Una función con tipo de retorno distinto de void lógico deberá retornar un valor compatible.
- return finaliza inmediatamente la ejecución de la función.
- Las variables locales podrán ocultar variables de ámbitos superiores sin modificarlas.

### 8.2 Ámbitos

```
Global
├── backend : server
├── replicas : int
├── configure() : function
│   └── Function Scope
│       ├── s : server
│       ├── replicas : int
│       └── If Scope
│           └── timeout : int
```

### 8.3 Recursividad

El intérprete deberá soportar recursividad directa. Se deberá implementar un límite de profundidad de llamadas configurable, con valor por defecto de 1000.

```
function countdown(int n) int {
    if (n <= 0) {
        return 0;
    }
    print(n);
    return countdown(n - 1);
}
```

## 9. Recursos de infraestructura

Los recursos son entidades especiales administradas por el entorno de ejecución. Sus propiedades tienen nombres y tipos predefinidos; no se permitirán propiedades adicionales.

### 9.1 Server

```
server backend {
    cpu = 4;
    memory = 8;
    disk = 100;
    os = "ubuntu";
}
```

| Propiedad | Tipo   | Restricción                               |
|-----------|--------|-------------------------------------------|
| cpu       | int    | > 0                                       |
| memory    | int    | > 0, GB                                   |
| disk      | int    | > 0, GB                                   |
| os        | string | no vacío                                  |
| status    | string | propiedad de solo lectura; inicia stopped |

### 9.2 Service

```
service api {
    port = 8080;
    replicas = 2;
}
```

| Propiedad | Tipo   | Restricción                    |
|-----------|--------|--------------------------------|
| port      | int    | 1..65535                       |
| replicas  | int    | >= 1                           |
| status    | string | solo lectura                   |
| host      | server | solo lectura después de deploy |

### 9.3 Database

```
database postgres {
    engine = "postgresql";
    version = "16";
    port = 5432;
}
```

| Propiedad | Tipo   | Restricción                         |
|-----------|--------|-------------------------------------|
| engine    | string | postgresql, mysql o sqlite simulado |
| version   | string | no vacío                            |
| port      | int    | 1..65535                            |
| status    | string | solo lectura                        |

## 10. Acciones nativas de AutoInfra

Las funciones nativas son implementadas por el intérprete y producen cambios en el estado simulado.

| Función    | Parámetros         | Retorno | Efecto                        |
|------------|--------------------|---------|-------------------------------|
| start      | resource           | bool    | Cambia stopped -> running     |
| stop       | resource           | bool    | Cambia running -> stopped     |
| restart    | resource           | bool    | Reinicia un recurso activo    |
| install    | server, string     | bool    | Agrega un paquete al servidor |
| uninstall  | server, string     | bool    | Elimina un paquete instalado  |
| deploy     | server, service    | bool    | Asocia y activa un servicio   |
| scale      | service, int       | bool    | Modifica réplicas             |
| connect    | resource, resource | bool    | Registra una conexión lógica  |
| disconnect | resource, resource | bool    | Elimina una conexión          |
| print      | valor...           | void    | Escribe en consola            |
| length     | array              | int     | Devuelve tamaño del arreglo   |
| status     | resource           | string  | Devuelve estado actual        |

### 10.1 Reglas de ejecución

- No se puede desplegar un servicio en un servidor detenido.
- No se puede instalar software en un servidor detenido.
- scale requiere al menos una réplica.
- start sobre un recurso ya activo no deberá duplicar efectos; retornará false y registrará advertencia.
- stop sobre un recurso detenido retornará false.
- connect no deberá crear conexiones duplicadas.
- Las acciones fallidas no deben dejar el entorno en un estado parcialmente modificado.

### 10.2 Ejemplo

```
start(backend);
install(backend, "docker");
install(backend, "nginx");

if (deploy(backend, api)) {
    scale(api, 3);
    print("API desplegada");
}
```

## 11. Tareas y automatización

Una task representa un procedimiento ejecutable sin parámetros, diseñado para expresar automatizaciones de infraestructura.

```
task deployProduction {
    start(backend);
    install(backend, "docker");

    if (backend.memory >= 8) {
        deploy(backend, api);
        scale(api, 3);
    }
}
```

### 11.1 Ejecución de tareas

```
main {
    run deployProduction;
}
```

Podrán existir múltiples instrucciones run. Las tareas se ejecutarán secuencialmente en el orden escrito.

### 11.2 Reutilización

Una task podrá invocar funciones y funciones nativas, pero no podrá recibir parámetros. Esta restricción diferencia las tareas de las funciones y simplifica el punto de entrada.

## 12. Dependencias y reglas de ejecución

Los servicios y bases de datos podrán declarar dependencias mediante una propiedad dependsOn de tipo arreglo de referencias.

```
service api {
    port = 8080;
    replicas = 2;
    dependsOn = [postgres];
}
```

Antes de desplegar api, todas sus dependencias deberán existir y encontrarse en estado running.

```
start(postgres);
start(backend);
deploy(backend, api);
```

### 12.1 Error por dependencia

```
INFRA-004
No se puede desplegar 'api'.
Dependencia 'postgres' se encuentra en estado stopped.
Línea: 28
Columna: 5
```

## 12.2 Atomicidad simulada

Cada función nativa deberá validar todas sus precondiciones antes de modificar el entorno. Si una precondición falla, no deberá aplicarse ningún cambio parcial.

## 13. Patrón Interpreter y AST

El patrón Interpreter es un requisito central. Cada construcción ejecutable del lenguaje deberá representarse mediante nodos del AST capaces de evaluar expresiones o ejecutar instrucciones.

```
class Expression {
  evaluate(environment) {
    throw new Error("evaluate() no implementado");
  }
}

class Instruction {
  execute(environment) {
    throw new Error("execute() no implementado");
  }
}
```

### 13.1 Nodos mínimos sugeridos

| Expresiones              | Instrucciones       |
|--------------------------|---------------------|
| LiteralExpression        | VariableDeclaration |
| IdentifierExpression     | Assignment          |
| BinaryExpression         | IfInstruction       |
| UnaryExpression          | WhileInstruction    |
| CallExpression           | ForInstruction      |
| PropertyAccessExpression | FunctionDeclaration |
| IndexExpression          | ReturnInstruction   |
| ArrayExpression          | TaskDeclaration     |
|                          | RunInstruction      |

### 13.2 Ejemplo de AST

```
int total = replicas * 2 + 1;

VariableDeclaration
├── Type: int
├── Name: total
└── Value: Binary(+)
    ├── Binary(*)
    │   ├── Identifier(replicas)
    │   └── Literal(2)
    └── Literal(1)
```

### 13.3 Resultado de ejecución

Se recomienda que Execute retorne una estructura capaz de propagar control de flujo (normal, break, continue, return) sin depender de excepciones.

```
const ExecSignal = Object.freeze({
  NORMAL: "normal",
  BREAK: "break",
  CONTINUE: "continue",
  RETURN: "return"
});
```

## 14. Análisis léxico, sintáctico y semántico

### 14.1 Analizador léxico

El lexer y parser deberán ejecutarse en JavaScript. Se permite utilizar ANTLR4 con target JavaScript, Jison, Chevrotain, Nearley u otra librería/generador equivalente. También se permite una implementación manual.

- Reconocer palabras reservadas, identificadores, números, cadenas, operadores y delimitadores.
- Registrar lexema, tipo de token, línea y columna.
- Procesar escapes mínimos en cadenas: \", \\, \n y \t.
- Reportar caracteres desconocidos y cadenas/comentarios sin cierre.

### 14.2 Analizador sintáctico

El parser deberá construir el AST. Se recomienda descenso recursivo y un esquema por niveles de precedencia para expresiones.

```
expression  -> logical_or
logical_or  -> logical_and ( "||" logical_and )*
logical_and -> equality ( "&&" equality )*
equality    -> comparison ( "==" | "!=" ) comparison )*
comparison  -> term ( ">" | ">=" | "<" | "<=" ) term )*
term        -> factor ( "+" | "-" ) factor )*
factor      -> unary ( "*" | "/" | "%" ) unary )*
unary       -> ("!" | "-") unary | postfix
postfix     -> primary ( call | property | index )*
```

### 14.3 Análisis semántico

A diferencia del proyecto de referencia, AutoInfra sí requiere validaciones semánticas porque existen tipos, ámbitos, funciones, recursos y reglas de dominio.

- Identificador declarado antes de su uso según reglas de alcance.
- No permitir declaraciones duplicadas en el mismo ámbito.
- Validar tipos de asignación y operandos.
- Validar firma de llamadas.
- Validar break/continue dentro de ciclos y return dentro de funciones.
- Validar propiedades existentes y de solo lectura.
- Validar referencias entre recursos.
- Validar restricciones numéricas de recursos.

### 14.4 Recuperación de errores

El parser deberá intentar continuar después de errores sintácticos utilizando puntos de sincronización como ';', '}' y palabras reservadas de declaración. El objetivo es reportar más de un error por ejecución cuando sea posible.

## 15. Punto de entrada

El archivo deberá contener exactamente un bloque main. Las declaraciones globales se registrarán antes de ejecutar sus instrucciones.

```
main {
    run prepareDatabase;
    run deployProduction;

    print("Automatización finalizada");
}
```

### 15.1 Orden de ejecución

9. Analizar léxicamente todo el archivo.
10. Construir el AST mediante el parser.
11. Ejecutar validaciones semánticas requeridas.
12. Registrar recursos, funciones y tareas globales.
13. Ejecutar el bloque main.
14. Generar bitácora y reportes finales.

### 15.2 Programa completo

```
server backend {
    cpu = 4;
    memory = 8;
    disk = 100;
    os = "ubuntu";
}

database postgres {
    engine = "postgresql";
    version = "16";
    port = 5432;
}

service api {
    port = 8080;
    replicas = 2;
    dependsOn = [postgres];
}

function enoughMemory(server s, int minimum) bool {
    return s.memory >= minimum;
}

task production {
    start(postgres);
    start(backend);
    install(backend, "docker");

    if (enoughMemory(backend, 8)) {
        deploy(backend, api);
        scale(api, 3);
    }
}

main {
    run production;
}
```

## 16. Entorno de trabajo

El estudiante deberá desarrollar una interfaz funcional para editar y ejecutar archivos .infra. El diseño visual queda a discreción del estudiante.

### 16.1 Funcionalidades mínimas

- Nuevo archivo .infra.
- Abrir archivo.
- Guardar y Guardar como.
- Ejecutar análisis e interpretación.
- Mostrar consola de salida.
- Mostrar listado de errores.
- Acceder a los reportes generados.
- Limpiar la ejecución actual sin conservar reportes anteriores.

### 16.2 Propuesta de distribución

|                               |                                                                              |
|-------------------------------|------------------------------------------------------------------------------|
| Archivo   Ejecutar   Reportes |                                                                              |
| Editor .infra                 | Estado de infraestructura<br>- servidores<br>- servicios<br>- bases de datos |
| Consola / errores / bitácora  |                                                                              |

### 16.3 Tecnología de interfaz

La interfaz será web. El backend del compilador/intérprete deberá ejecutarse sobre JavaScript en Node.js y el frontend podrá utilizar React, Vue, Angular, Svelte u otro framework equivalente.

## 17. Reportes

Los reportes deberán regenerarse en cada ejecución y corresponder únicamente al archivo analizado en ese momento.

### 17.1 Tabla de tokens

| # | Lexema  | Token      | Línea | Columna |
|---|---------|------------|-------|---------|
| 1 | server  | SERVER     | 1     | 1       |
| 2 | backend | IDENTIFIER | 1     | 8       |
| 3 | {       | LBACE      | 1     | 16      |

### 17.2 Reporte de errores

| # | Tipo            | Código    | Descripción                     | Línea | Columna |
|---|-----------------|-----------|---------------------------------|-------|---------|
| 1 | Semántico       | SEM-003   | Variable 'x' no declarada       | 12    | 9       |
| 2 | Infraestructura | INFRA-004 | Dependencia 'postgres' detenida | 28    | 5       |

### 17.3 Tabla de símbolos

| Identificador | Categoría | Tipo   | Ámbito    | Valor/Estado |
|---------------|-----------|--------|-----------|--------------|
| backend       | recurso   | server | global    | running      |
| replicas      | variable  | int    | global    | 3            |
| s             | parámetro | server | canDeploy | backend      |

### 17.4 AST

Se deberá generar una representación gráfica del AST utilizando Graphviz o una alternativa equivalente. El reporte debe mostrar el tipo de nodo y los valores relevantes.

### 17.5 Estado final de infraestructura

| Recurso  | Tipo     | Estado  | Detalles                 |
|----------|----------|---------|--------------------------|
| backend  | server   | running | 4 CPU, 8 GB, docker      |
| postgres | database | running | postgresql 16 :5432      |
| api      | service  | running | host=backend, replicas=3 |

### 17.6 Bitácora

```
[OK] postgres -> running
[OK] backend -> running
[OK] docker installed on backend
[OK] api deployed on backend
[OK] api scaled from 2 to 3 replicas
```

## 18. Requerimientos mínimos

Para tener derecho a calificación, el proyecto deberá cumplir como mínimo con:

- Proyecto instalable y ejecutable con Node.js/JavaScript.
- Interfaz funcional.
- Apertura, edición y guardado de archivos .infra.
- Lexer manual funcional.
- Parser manual que construya AST.
- Aplicación comprobable del patrón Interpreter.
- Variables, expresiones aritméticas, relacionales y lógicas.
- if/else, while y for.
- Funciones con parámetros y return.
- Tabla de símbolos con ámbitos.
- Al menos server y service funcionales.
- Al menos seis funciones nativas de infraestructura.
- Reporte de tokens, errores, tabla de símbolos y AST.
- Documentación completa.

## 19. Ponderación

| Evaluación                     | Puntos |
|--------------------------------|--------|
| Archivo básico                 | 15     |
| Archivo intermedio             | 25     |
| Archivo avanzado               | 35     |
| Preguntas sobre implementación | 10     |
| Reportes                       | 8      |
| Manuales y gramática           | 7      |
| Total                          | 100    |

La nota obtenida sobre 100 se convertirá proporcionalmente al valor de 45 puntos asignado al proyecto dentro del curso. [Puede llegar a cambiar](#)

### 19.1 Niveles de archivo

| Nivel      | Características esperadas                                                                   |
|------------|---------------------------------------------------------------------------------------------|
| Básico     | Recursos, variables, operaciones, if/else, funciones nativas simples.                       |
| Intermedio | Ciclos, funciones, arreglos, scopes, tasks, acceso a propiedades.                           |
| Avanzado   | Dependencias, recursividad, recuperación de errores, casos combinados y reportes completos. |

### 21.1 Estructura sugerida del proyecto JavaScript

```

autoinfra/
├── backend/
│   ├── src/
│   │   ├── grammar/      # .g4 si se usa ANTLR
│   │   ├── generated/    # lexer/parser generados
│   │   ├── ast/
│   │   ├── semantic/
│   │   ├── interpreter/
│   │   ├── environment/
│   │   ├── infrastructure/
│   │   ├── reports/
│   │   └── api/
│   └── package.json
├── frontend/
│   └── src/

```

```
├── package.json
├── examples/
└── README.md
```

## 21.2 Nomenclatura del repositorio

OLC1\_Proyecto2\_#Carnet

## 22. Casos de prueba sugeridos

Los siguientes casos describen el tipo de cobertura que deberá soportar el proyecto. Los archivos oficiales de calificación podrán combinar estas características.

### 22.1 Archivo básico

```
server web {
    cpu = 2;
    memory = 4;
    disk = 50;
    os = "ubuntu";
}

service frontend {
    port = 80;
    replicas = 1;
}

main {
    start(web);

    int cost = web.cpu * 10 + web.memory * 5;

    if (cost >= 40) {
        install(web, "nginx");
        deploy(web, frontend);
    }
}
```

### 22.2 Archivo intermedio

```
server node {
    cpu = 4;
    memory = 8;
    disk = 100;
    os = "ubuntu";
}

string[] packages = ["docker", "git", "nginx"];

function prepare(server s, string[] pkgs) bool {
    int i = 0;

    while (i < length(pkgs)) {
        install(s, pkgs[i]);
        i = i + 1;
    }

    return true;
}

task setup {
    start(node);
    prepare(node, packages);
}

main {
    run setup;
}
```

## 22.3 Archivo avanzado

```
database db {
    engine = "postgresql";
    version = "16";
    port = 5432;
}

server backend {
    cpu = 8;
    memory = 16;
    disk = 200;
    os = "ubuntu";
}

service api {
    port = 8080;
    replicas = 2;
    dependsOn = [db];
}

function retryStart(server s, int n) bool {
    if (n <= 0) {
        return false;
    }

    if (start(s)) {
        return true;
    }

    return retryStart(s, n - 1);
}

task production {
    start(db);
    retryStart(backend, 3);

    if (backend.status == "running" && db.status == "running") {
        install(backend, "docker");
        deploy(backend, api);

        for (int i = 2; i <= 5; i = i + 1) {
            scale(api, i);
        }
    }
}

main {
    run production;
}
```

## 22.4 Casos de error obligatorios

| Caso                  | Ejemplo                     | Resultado esperado                        |
|-----------------------|-----------------------------|-------------------------------------------|
| Léxico                | int x = 10 @ 2;             | Carácter no reconocido con línea/columna. |
| Sintáctico            | if (x > 2 {                 | Error y sincronización.                   |
| Variable no declarada | x = 5;                      | Error semántico.                          |
| Tipo incompatible     | int x = "hola";             | Error semántico.                          |
| División por cero     | 10 / 0                      | Error de ejecución controlado.            |
| Propiedad inexistente | backend.coresX              | Error semántico.                          |
| Solo lectura          | backend.status = "running"; | Error semántico.                          |
| Dependencia detenida  | deploy(backend, api)        | INFRA-004.                                |
| break inválido        | break; fuera de ciclo       | Error semántico.                          |
| Firma incorrecta      | scale(api)                  | Error semántico.                          |

## 22.5 Salida esperada de ejemplo

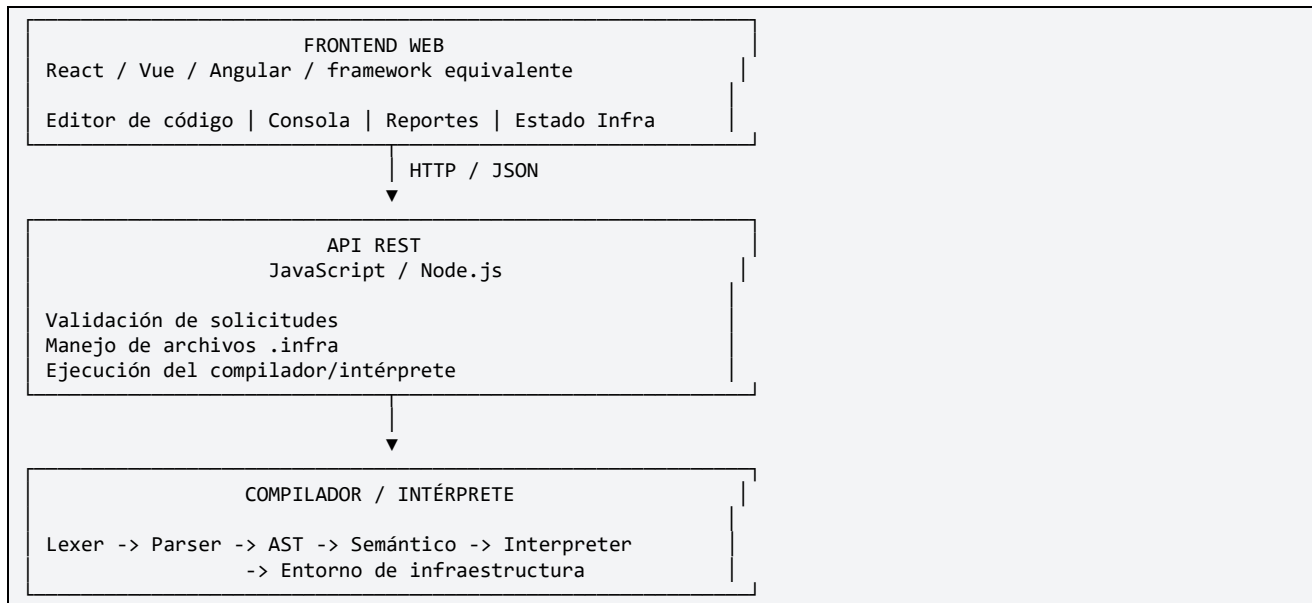
```
=====
AUTOINFRA - EXECUTION
=====
[OK] Resource 'db' registered
[OK] Resource 'backend' registered
[OK] Resource 'api' registered
[OK] db started
[OK] backend started
[OK] docker installed on backend
[OK] api deployed on backend
[OK] api scaled to 5 replicas

FINAL STATE
-----
backend : server      : running
db       : database   : running
api      : service    : running (5 replicas)
=====
```

## 24. Arquitectura cliente-servidor e interfaz gráfica

Además del intérprete desarrollado en JavaScript/Node.js, el proyecto deberá incluir una interfaz gráfica web. La aplicación se dividirá en un backend encargado del compilador/intérprete y un frontend encargado de la edición, envío de código, visualización de resultados y consulta de reportes.

### 24.1 Arquitectura general



### 24.2 Backend obligatorio en JavaScript/Node.js

Todo el núcleo del compilador deberá implementarse en JavaScript y ejecutarse sobre Node.js. El backend expondrá una API REST que permita al frontend enviar código fuente y obtener los resultados del análisis y de la ejecución.

- El lexer, parser, AST, tabla de símbolos, análisis semántico e Interpreter deberán residir en el backend JavaScript/Node.js.
- El frontend no deberá implementar reglas léxicas, sintácticas ni semánticas.
- La API deberá responder utilizando JSON.
- Los errores deberán devolverse de forma estructurada con tipo, descripción, línea y columna.
- La ejecución de infraestructura será completamente simulada y permanecerá dentro del backend.
- El backend podrá utilizar Express, Fastify, Koa, NestJS u otro framework HTTP compatible con Node.js.

### 24.3 Frontend

El frontend deberá desarrollarse utilizando React, Vue, Angular, Svelte u otro framework web moderno. React será la tecnología recomendada para los ejemplos del enunciado.

- Editor de código fuente para archivos .infra.
- Botones para Nuevo, Abrir, Guardar y Ejecutar.
- Consola de salida del intérprete.
- Panel de errores léxicos, sintácticos, **semánticos y** de infraestructura.
- Tabla de tokens.

- Tabla de símbolos.
- Vista del estado final de servidores, servicios y bases de datos.
- Vista del AST generado.
- Bitácora de acciones ejecutadas.
- Indicadores visuales de ejecución exitosa o fallida.

## 24.4 Endpoints mínimos sugeridos

| Método | Endpoint     | Finalidad                                                                 |
|--------|--------------|---------------------------------------------------------------------------|
| POST   | /api/analyze | Realiza análisis léxico, sintáctico y semántico sin ejecutar.             |
| POST   | /api/execute | Analiza y ejecuta el archivo .infra.                                      |
| POST   | /api/tokens  | Devuelve la tabla de tokens reconocidos.                                  |
| POST   | /api/ast     | Devuelve información del AST o un formato serializable para visualizarlo. |
| POST   | /api/symbols | Devuelve la tabla de símbolos resultante.                                 |
| POST   | /api/errors  | Devuelve los errores encontrados en el código.                            |
| POST   | /api/format  | Opcional: devuelve código con formato normalizado.                        |
| GET    | /api/health  | Verifica que el backend esté disponible.                                  |

## 24.5 Ejemplo de solicitud

POST /api/execute  
Content-Type: application/json

```
{
  "source": "server backend { cpu = 4; memory = 8; disk = 100; os = \"ubuntu\"; } main { start(backend); }"
```

## 24.6 Ejemplo de respuesta

```
{
  "success": true,
  "console": [
    "[OK] Resource 'backend' registered",
    "[OK] backend started"
  ],
  "errors": [],
  "tokens": [],
  "symbols": [],
  "infrastructure": {
    "servers": [
      {
        "name": "backend",
        "status": "running",
        "cpu": 4,
        "memory": 8,
        "disk": 100,
        "os": "ubuntu"
      }
    ],
    "services": [],
    "databases": []
  }
}
```

## 24.7 Componentes sugeridos del frontend

| Componente   | Responsabilidad                                       |
|--------------|-------------------------------------------------------|
| CodeEditor   | Edición del código .infra.                            |
| Toolbar      | Nuevo, abrir, guardar y ejecutar.                     |
| ConsolePanel | Salida producida por print y bitácora del intérprete. |

|                    |                                                 |
|--------------------|-------------------------------------------------|
| ErrorsPanel        | Errores con tipo, línea, columna y descripción. |
| TokensTable        | Visualización de tokens reconocidos.            |
| SymbolsTable       | Visualización de tabla de símbolos.             |
| InfrastructureView | Estado actual/final de recursos simulados.      |
| AstView            | Representación visual del AST.                  |
| ExecutionStatus    | Estado de ejecución y tiempo aproximado.        |

## 24.8 Comunicación frontend-backend

Al presionar Ejecutar, el frontend deberá enviar el contenido completo del editor al backend. El backend procesará el código y devolverá una única respuesta con los resultados necesarios para actualizar la consola, errores, reportes y estado de infraestructura.

```

Usuario presiona Ejecutar
↓
Frontend obtiene contenido del editor
↓
POST /api/execute
↓
Backend JavaScript/Node.js:
Lexer
↓
Parser
↓
AST
↓
Análisis semántico
↓
Interpreter
↓
JSON de respuesta
↓
Frontend actualiza:
Consola + Errores + Tokens + Símbolos + AST + Infraestructura

```

## 24.9 Persistencia de archivos

La aplicación no requiere una base de datos. Abrir y guardar archivos puede realizarse desde el navegador o mediante endpoints del backend. Si se decide implementar almacenamiento en el servidor, deberá limitarse a archivos .infra y no será parte obligatoria de la evaluación.

## 24.10 CORS y ejecución local

El backend deberá permitir el consumo desde el origen del frontend durante el desarrollo. Se recomienda ejecutar ambos componentes de manera independiente, por ejemplo:

```

Frontend:
http://localhost:5173

Backend:
http://localhost:8080

```

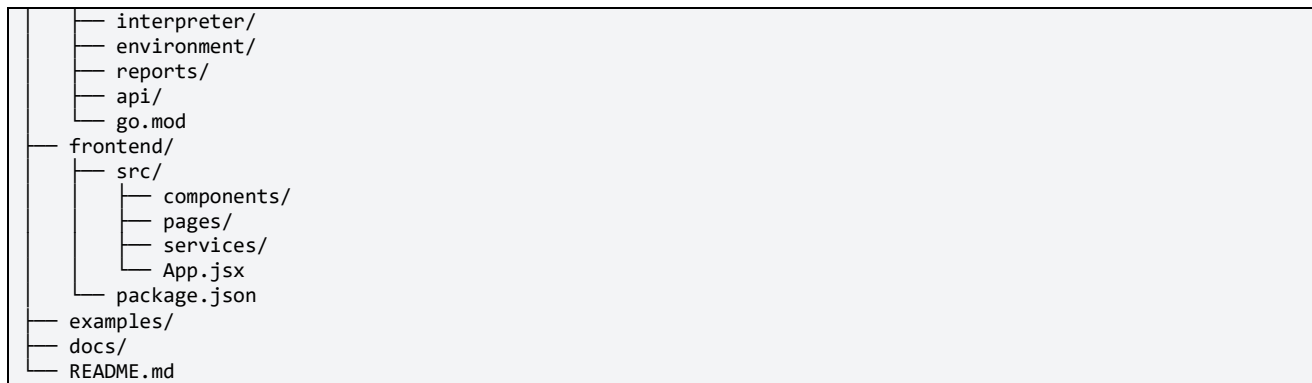
El estudiante deberá documentar los comandos necesarios para iniciar ambos componentes.

## 24.11 Estructura sugerida del repositorio

```

OLC1_Proyecto2_#Carnet/
├── backend/
│   ├── cmd/
│   ├── lexer/
│   ├── parser/
│   ├── ast/
│   └── semantic/

```



### 24.12 Requerimientos mínimos de la interfaz

- El frontend deberá consumir al menos el endpoint de ejecución del backend.
- La interfaz no podrá ser únicamente una consola de terminal.
- Se deberá visualizar el código fuente, la salida y los errores en áreas diferenciadas.
- Los reportes deberán ser consultables desde la interfaz.
- El estado final de la infraestructura deberá presentarse en tarjetas, tablas, árbol o visualización equivalente.
- El frontend deberá indicar claramente cuándo una ejecución terminó correctamente.
- La aplicación deberá poder ejecutarse localmente con instrucciones documentadas.

### 24.13 Restricciones adicionales

- El compilador deberá ejecutarse realmente en el backend JavaScript/Node.js; no se permite delegar el análisis a servicios externos.
- No se permite traducir el archivo `.infra` a JavaScript y ejecutarlo mediante `eval()`, `Function()`, `vm` sin aislamiento o mecanismos equivalentes. El código debe ejecutarse recorriendo el AST mediante el patrón `Interpreter`.
- No se permite delegar análisis sintáctico o semántico al frontend.
- El frontend puede utilizar librerías de edición como Monaco Editor o CodeMirror.
- El frontend puede utilizar librerías de visualización para el AST, siempre que la estructura del AST sea generada por el backend.
- No se requiere autenticación, usuarios, base de datos ni despliegue en la nube.

### 24.14 Actualización de requerimientos mínimos

Para tener derecho a calificación también será obligatorio que backend y frontend puedan iniciarse correctamente y comunicarse entre sí. Una solución que ejecute el intérprete únicamente desde terminal no cumplirá con los requerimientos mínimos del Proyecto 2.

## 25. Herramientas permitidas para el análisis

El proyecto se desarrollará en JavaScript. Se permite utilizar herramientas que generen código para el análisis léxico y sintáctico. La herramienta recomendada es ANTLR4 utilizando el target JavaScript, aunque el estudiante podrá utilizar otra librería equivalente.

| Herramienta         | Permitido | Uso esperado                              | Observación                                                            |
|---------------------|-----------|-------------------------------------------|------------------------------------------------------------------------|
| ANTLR4              | Sí        | Generar lexer, parser, listener o visitor | La gramática .g4 y el código generado deben formar parte del proyecto. |
| Jison               | Sí        | Generación de lexer/parser                | Debe construirse un AST propio.                                        |
| Chevrotain          | Sí        | Lexer/parser programático                 | El AST e Interpreter siguen siendo obligatorios.                       |
| Nearley             | Sí        | Parser generado                           | Debe documentarse la gramática utilizada.                              |
| Parser manual       | Sí        | Descenso recursivo u otra técnica         | Debe respetar precedencia y asociatividad.                             |
| eval() / Function() | No        | Ejecutar directamente código traducido    | La ejecución debe realizarse recorriendo el AST.                       |

### 25.1 Uso esperado de ANTLR

```

Código .infra
↓
AutoInfraLexer.g4
↓
Tokens
↓
AutoInfraParser.g4
↓
Parse Tree
↓
Visitor / Listener
↓
AST propio
↓
Análisis semántico
↓
Interpreter
↓
Estado de infraestructura

```

El Parse Tree generado por ANTLR no sustituye al AST. El estudiante deberá transformar la estructura generada por la herramienta en nodos propios del lenguaje AutoInfra.

## 26. Operaciones permitidas

Las siguientes tablas definen de forma explícita las combinaciones válidas de operandos. Toda combinación no incluida deberá reportarse como error semántico.

### 26.1 Operadores aritméticos

| Operador | Operandos permitidos               | Resultado                | Ejemplo                                  |
|----------|------------------------------------|--------------------------|------------------------------------------|
| +        | int + int                          | int                      | int total = 10 + 5; // 15                |
| +        | int/float + int/float              | float si participa float | float cpu = 2.5 + 1; // 3.5              |
| +        | string + cualquier valor primitivo | string                   | string node = "worker-" + 3; // worker-3 |
| -        | int/float - int/float              | numérico                 | int free = 16 - 8; // 8                  |
| *        | int/float * int/float              | numérico                 | int capacity = 4 * 8; // 32              |
| /        | int/float / int/float              | float                    | float ratio = 5 / 2; // 2.5              |

|   |           |     |                         |
|---|-----------|-----|-------------------------|
| % | int % int | int | int shard = 7 % 3; // 1 |
|---|-----------|-----|-------------------------|

La división entre cero deberá producir un error de ejecución controlado. El operador % solo acepta enteros.

## 26.2 Operadores relacionales

| Operador | Operandos permitidos    | Resultado | Ejemplo             |
|----------|-------------------------|-----------|---------------------|
| <        | int/float con int/float | bool      | backend.memory < 16 |
| <=       | int/float con int/float | bool      | api.replicas <= 5   |
| >        | int/float con int/float | bool      | backend.cpu > 2     |
| >=       | int/float con int/float | bool      | backend.disk >= 100 |

## 26.3 Igualdad y desigualdad

| Operador | Operandos permitidos    | Resultado | Ejemplo                  |
|----------|-------------------------|-----------|--------------------------|
| ==       | números compatibles     | bool      | backend.memory == 8      |
| !=       | números compatibles     | bool      | api.port != 3000         |
| == / !=  | string con string       | bool      | backend.os == "ubuntu"   |
| == / !=  | bool con bool           | bool      | active == true           |
| == / !=  | recursos del mismo tipo | bool      | backend == primaryServer |

## 26.4 Operadores lógicos

| Operador | Operandos permitidos | Resultado | Ejemplo                                             |
|----------|----------------------|-----------|-----------------------------------------------------|
| &&       | bool && bool         | bool      | backend.memory >= 8 && backend.status == "running"  |
|          | bool    bool         | bool      | api.status == "running"    api.status == "degraded" |
| !        | !bool                | bool      | !maintenanceMode                                    |

Los operadores && y || deberán utilizar corto circuito.

## 26.5 Operadores unarios

| Operador | Operando    | Resultado | Ejemplo                            |
|----------|-------------|-----------|------------------------------------|
| -        | int o float | numérico  | int offset = -5;                   |
| !        | bool        | bool      | bool available = !maintenanceMode; |

## 26.6 Compatibilidad de asignación

| Destino  | Valores permitidos  | Ejemplo válido                  | Ejemplo inválido            |
|----------|---------------------|---------------------------------|-----------------------------|
| int      | int                 | int replicas = 3;               | int replicas = 3.5;         |
| float    | int o float         | float limit = 2;                | float limit = "2";          |
| string   | string              | string env = "prod";            | string env = true;          |
| bool     | bool                | bool active = true;             | bool active = 1;            |
| server   | server              | server target = backend;        | server target = api;        |
| service  | service             | service current = api;          | service current = postgres; |
| database | database            | database current = postgres;    | database current = backend; |
| T[]      | arreglo del mismo T | string[] p = ["git", "docker"]; | string[] p = ["git", 5];    |

## 27. Propiedades disponibles por recurso

Las propiedades se consultan mediante la sintaxis recurso.propiedad. Las propiedades marcadas como solo lectura son actualizadas exclusivamente por el intérprete y las funciones nativas.

## 27.1 Propiedades de server

| Propiedad | Tipo     | Acceso       | Ejemplo                                                          |
|-----------|----------|--------------|------------------------------------------------------------------|
| cpu       | int      | Lectura      | if (backend.cpu >= 4) {<br>print("CPU suficiente"); }            |
| memory    | int      | Lectura      | if (backend.memory >= 8) {<br>deploy(backend, api); }            |
| disk      | int      | Lectura      | bool enoughDisk =<br>backend.disk >= 100;                        |
| os        | string   | Lectura      | if (backend.os == "ubuntu") {<br>install(backend, "docker"); }   |
| status    | string   | Solo lectura | if (backend.status == "running")<br>{ print("Backend activo"); } |
| packages  | string[] | Solo lectura | int installed =<br>length(backend.packages);                     |

## 27.2 Propiedades de service

| Propiedad | Tipo       | Acceso       | Ejemplo                                                            |
|-----------|------------|--------------|--------------------------------------------------------------------|
| port      | int        | Lectura      | if (api.port == 8080) {<br>print("Puerto correcto"); }             |
| replicas  | int        | Lectura      | if (api.replicas < 3) { scale(api,<br>3); }                        |
| status    | string     | Solo lectura | if (api.status == "running") {<br>print("API disponible"); }       |
| host      | server     | Solo lectura | if (api.host == backend) {<br>print("Desplegada en<br>backend"); } |
| dependsOn | resource[] | Lectura      | int dependencies =<br>length(api.dependsOn);                       |

## 27.3 Propiedades de database

| Propiedad | Tipo   | Acceso       | Ejemplo                                                                 |
|-----------|--------|--------------|-------------------------------------------------------------------------|
| engine    | string | Lectura      | if (postgres.engine ==<br>"postgresql") { print("Motor<br>correcto"); } |
| version   | string | Lectura      | bool modern =<br>postgres.version == "16";                              |
| port      | int    | Lectura      | if (postgres.port == 5432) {<br>print("Puerto PostgreSQL"); }           |
| status    | string | Solo lectura | if (postgres.status != "running")<br>{ start(postgres); }               |

## 27.4 Valores posibles de status

| Recurso  | Estados mínimos                                   | Ejemplo de consulta          |
|----------|---------------------------------------------------|------------------------------|
| server   | stopped, running, failed                          | backend.status == "running"  |
| database | stopped, running, failed                          | postgres.status == "stopped" |
| service  | undeployed, running, stopped, degraded,<br>failed | api.status == "degraded"     |

status es una propiedad de solo lectura. Por ejemplo, backend.status = "running" deberá generar error semántico. El cambio correcto se realiza mediante start(backend).

## 28. Catálogo de expresiones con ejemplos

Toda forma de expresión propuesta para AutoInfra deberá poder observarse en el código fuente y evaluarse mediante nodos del AST. A continuación se define un ejemplo mínimo de cada categoría.

| Expresión           | Ejemplo                                               | Resultado / finalidad                                     |
|---------------------|-------------------------------------------------------|-----------------------------------------------------------|
| Literal entero      | 10                                                    | Valor int 10.                                             |
| Literal decimal     | 2.5                                                   | Valor float 2.5.                                          |
| Literal string      | "production"                                          | Cadena production.                                        |
| Literal booleano    | true                                                  | Valor bool verdadero.                                     |
| Identificador       | replicas                                              | Consulta el valor de una variable en el entorno.          |
| Agrupación          | (2 + 3) * 4                                           | Primero evalúa 2 + 3.                                     |
| Aritmética          | backend.cpu * 2 + backend.memory                      | Combina propiedades y operadores numéricos.               |
| Concatenación       | "worker-" + i                                         | Genera nombres dinámicos como worker-2.                   |
| Relacional          | backend.memory >= 8                                   | Produce bool.                                             |
| Igualdad string     | backend.status == "running"                           | Comprueba el estado del servidor.                         |
| Lógica AND          | backend.status == "running" && backend.memory >= 8    | Ambas condiciones deben ser verdaderas.                   |
| Lógica OR           | api.status == "running"    api.status == "degraded"   | Acepta cualquiera de los dos estados.                     |
| Negación            | !maintenanceMode                                      | Invierte un bool.                                         |
| Acceso a propiedad  | backend.status                                        | Lee una propiedad del recurso.                            |
| Arreglo literal     | ["docker", "git", "nginx"]                            | Crea string[].                                            |
| Índice de arreglo   | packages[0]                                           | Devuelve el primer elemento.                              |
| Llamada de función  | enoughMemory(backend, 8)                              | Evalúa una función definida por el usuario.               |
| Llamada nativa      | status(backend)                                       | Devuelve el estado como string.                           |
| Función anidada     | length(api.dependsOn) > 0                             | Usa el resultado de una función dentro de otra expresión. |
| Expresión compuesta | (backend.cpu * 2 >= 8) && backend.status == "running" | Combina agrupación, aritmética, comparación y lógica.     |

## 28.1 Ejemplo de status en una condición

```
server backend {
    cpu = 4;
    memory = 8;
    disk = 100;
    os = "ubuntu";
}

main {
    // Un server inicia en estado stopped.
    if (backend.status == "stopped") {
        start(backend);
    }

    // start modifica internamente la propiedad status.
    if (backend.status == "running") {
        print("El servidor ya puede recibir despliegues");
    }
}
```

## 28.2 Ejemplo combinado con service.status y host

```
main {
    start(backend);
    deploy(backend, api);

    if (api.status == "running" && api.host == backend) {
        print("API activa en el servidor backend");
    }
}
```

## 28.3 Ejemplo con database.status y dependencia

```
task deployProduction {
    if (postgres.status != "running") {
```

```

        start(postgres);
    }

    if (backend.status != "running") {
        start(backend);
    }

    if (postgres.status == "running" && backend.status == "running") {
        deploy(backend, api);
    }
}

```

## 28.4 Ejemplo con arreglos e índices

```

string[] packages = ["docker", "git", "nginx"];

main {
    start(backend);

    for (int i = 0; i < length(packages); i = i + 1) {
        install(backend, packages[i]);
    }

    print("Paquetes instalados: " + length(backend.packages));
}

```

## 28.5 Ejemplo con función definida por el usuario

```

function canDeploy(server s, int minCpu, int minMemory) bool {
    return s.status == "running"
        && s.cpu >= minCpu
        && s.memory >= minMemory;
}

main {
    start(backend);

    if (canDeploy(backend, 4, 8)) {
        deploy(backend, api);
    }
}

```

## 28.6 Ejemplo de corto circuito

```

// Si backend no está running, no es necesario evaluar la segunda expresión.
if (backend.status == "running" && backend.memory >= 8) {
    deploy(backend, api);
}

```

## 29. Funciones nativas: ejemplos obligatorios

| Función                   | Ejemplo de uso                | Resultado esperado                                 |
|---------------------------|-------------------------------|----------------------------------------------------|
| start(resource)           | start(backend);               | backend.status cambia de stopped a running.        |
| stop(resource)            | stop(backend);                | backend.status cambia a stopped.                   |
| restart(resource)         | restart(backend);             | Reinicia el recurso y retorna true si fue posible. |
| install(server, string)   | install(backend, "docker");   | Agrega docker a backend.packages.                  |
| uninstall(server, string) | uninstall(backend, "docker"); | Elimina docker de backend.packages.                |
| deploy(server, service)   | deploy(backend, api);         | api.host = backend y api.status = running.         |

|                                |                                     |                                                         |
|--------------------------------|-------------------------------------|---------------------------------------------------------|
| scale(service, int)            | scale(api, 4);                      | api.replicas pasa a 4.                                  |
| connect(resource, resource)    | connect(api, postgres);             | Registra una conexión lógica.                           |
| disconnect(resource, resource) | disconnect(api, postgres);          | Elimina la conexión lógica.                             |
| print(valor...)                | print("Estado: " + backend.status); | Escribe en la consola del frontend.                     |
| length(array)                  | length(backend.packages)            | Retorna cantidad de elementos.                          |
| status(resource)               | status(backend)                     | Retorna el mismo valor consultable como backend.status. |

## 30. Ejemplo integral de AutoInfra

```

server backend {
    cpu = 4;
    memory = 8;
    disk = 100;
    os = "ubuntu";
}

database postgres {
    engine = "postgresql";
    version = "16";
    port = 5432;
}

service api {
    port = 8080;
    replicas = 2;
    dependsOn = [postgres];
}

string[] packages = ["docker", "git"];

function ready(server s, database db) bool {
    return s.status == "running"
        && db.status == "running"
        && s.memory >= 8;
}

task production {
    start(postgres);
    start(backend);

    for (int i = 0; i < length(packages); i = i + 1) {
        install(backend, packages[i]);
    }

    if (ready(backend, postgres)) {
        deploy(backend, api);
        scale(api, 3);
    }

    if (api.status == "running") {
        print("API disponible en puerto " + api.port);
        print("Servidor: " + api.host.os);
        print("Réplicas: " + api.replicas);
    }
}

main {
    run production;
}

```

Este ejemplo integra declaraciones de recursos, arreglos, funciones, task, ciclo for, llamadas nativas, propiedades, status, operadores aritméticos/relacionales/lógicos y ejecución desde main.

## 31. Tecnologías definitivas

| Componente             | Tecnología / regla                                                                                            |
|------------------------|---------------------------------------------------------------------------------------------------------------|
| Backend del compilador | JavaScript ejecutado con Node.js.                                                                             |
| Lexer / Parser         | ANTLR4 target JavaScript o cualquier librería/generador equivalente; implementación manual también permitida. |
| AST                    | Estructura propia obligatoria.                                                                                |
| Interpreter            | Implementación propia mediante clases/objetos JavaScript.                                                     |
| API                    | REST/JSON con Express, Fastify, Koa, NestJS o equivalente.                                                    |
| Frontend               | React recomendado; Vue, Angular, Svelte u otro framework equivalente permitido.                               |
| Editor                 | Monaco Editor, CodeMirror u otro componente permitido.                                                        |
| Visualización AST      | Graphviz, Cytoscape, React Flow, D3 u otra librería permitida.                                                |
| Persistencia           | No requerida.                                                                                                 |

## Entregables

El estudiante deberá entregar los siguientes elementos:

- Código fuente completo del proyecto.
- Último hash de Git realizado antes de la fecha límite de entrega.
- Manual de usuario en formato Markdown (.md).
  - Debe incluir capturas de pantalla que expliquen detalladamente el funcionamiento del entorno de trabajo.
  - Debe mostrar el proceso para crear, abrir, guardar y ejecutar archivos.
  - Debe incluir capturas de los reportes generados por la aplicación.
- Manual técnico en formato Markdown (.md).
  - Debe incluir la información necesaria para facilitar el mantenimiento futuro del proyecto.
  - Debe especificar el lenguaje de programación utilizado.
  - Debe indicar las herramientas y librerías empleadas.
  - Debe describir la estructura general del proyecto.
  - Debe explicar las clases, métodos y funciones más importantes.
  - Debe explicar el funcionamiento del analizador léxico, analizador sintáctico y motor de ejecución.

- Archivo de gramática en formato Markdown (.md).
  - Debe contener la gramática completa del lenguaje.

## Restricciones

- La entrega deberá realizarse mediante la plataforma UEDI, enviando el enlace del repositorio privado de GitHub donde se encuentre alojado el proyecto.
- El repositorio deberá nombrarse utilizando el siguiente formato:

OLC1\_Proyecto2\_#Carnet

Ejemplo:

OLC1\_Proyecto2\_202012345

- Se deberá agregar al auxiliar encargado como colaborador del repositorio de GitHub.
- El auxiliar deberá contar con permisos de desarrollador (Developer).

## Usuarios de GitHub por sección

- **Sección B — 002Fer**
- **Sección C — JohnAlds**
- **Sección N — LempDnote**

La entrega deberá realizarse mediante la plataforma **UEDI**, enviando el enlace del repositorio privado de GitHub.

## Penalizaciones

- 50% por entrega tarde en UEDI
- 50% al no comprobar autoría
- 15% al no enviar invitación en github

**No se calificarán entregas realizadas después de la fecha y hora establecidas.**